

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING



OBJECT-ORIENTED PROGRAMMING
LAB EXERCISE 2

PRACTICAL CONTENT

SUBMISSION REQUIREMENTS AND INSTRUCTIONS

Submission instructions:

- Submit a .PDF file named: STUDENTID_LABX.PDF (X is the lab session number; for example: 18520412_LAB2.PDF).
- The .PDF file must follow the template provided on Courses.
- Take screenshots of the source code and place them in the template. For each method, clearly state the input, output, and the algorithmic approach used to solve the problem.
- Upload the source code to Drive and include the sharing link in the report template.

Note: Submissions that do not satisfy the above requirements will not be graded.

LAB 2: FROM OBJECTS TO CLASSES, METHODS, CONSTRUCTORS, AND OVERLOADING

This lab introduces the first complete object-oriented programming workflow in Java. Students learn how to move from a real-world problem to an object model, define a class with attributes and methods, control member accessibility using Java access modifiers, create and initialize objects using constructors (including a user-defined copy constructor), and finally use method and constructor overloading when one operation needs several valid forms.

Learning Outcomes

- Identify candidate objects from a simple problem description.
- Translate an object into a Java class with appropriate attributes and methods.
- Create objects and access their state and behavior.
- Define no-argument, parameterized, and user-defined copy constructors and use this keyword correctly.
- Use public, private, protected, and package-private access appropriately in simple class designs.
- Explain and implement method overloading and constructor overloading.
- Apply the complete workflow to small OOP problems.

1. From a Problem to an Object

Before writing a class, first ask: “What thing in the problem has its own data and behavior?” In OOP, that thing is usually a candidate object. A useful beginner rule is: nouns often suggest objects or attributes, while verbs often suggest methods.

Example problem: “A university needs to store each student’s name, student ID, and GPA. A student can display personal information and determine whether the current GPA satisfies a passing threshold.”

Step 1 - Identify the main object: Student.

Step 2 - Identify data that describes the object: name, studentId, GPA.

IT002 – Object-Oriented Programming

Step 3 - Identify behaviors the object should perform: display information, check passing status.

Problem element	OOP concept	Java example
Student	Object / Class	Student
name, student ID, GPA	Attributes	String name; String studentId; double gpa;
display information	Method	void displayInfo()
check passing status	Method	boolean isPassed()

Key idea: Do not start by writing methods randomly. First determine what object you are modeling, what information it owns, and what responsibilities it should have.

2. Building a Java Class

A class is a blueprint for creating objects. For this lab, a class will mainly contain three parts: attributes (state), methods (behavior), and constructors (initialization).

```
class Student {  
    // 1. Attributes  
    String name;  
    String studentId;  
    double gpa;  
  
    // 2. Methods  
    void displayInfo() {  
        System.out.println(studentId + " - " + name + " - GPA: " + gpa);  
    }  
}
```

The class above defines what every Student object can store and what it can do. It does not yet represent one specific student; an object must be created from the class.

2.1 Attributes: Representing Object State

Attributes (also called fields or instance variables) store the state of an object. Each object created from the same class has its own values for these attributes.

```
class Student {  
    String name;  
    String studentId;  
    double gpa;  
}
```

Creating two objects shows that the same class can produce different object states:

IT002 – Object-Oriented Programming

```
Student s1 = new Student();
s1.name = "An";
s1.studentId = "23520001";
s1.gpa = 8.2;

Student s2 = new Student();
s2.name = "Binh";
s2.studentId = "23520002";
s2.gpa = 6.7;

System.out.println(s1.name); // An
System.out.println(s2.name); // Binh
```

Remember: A class defines the structure. An object stores concrete values in memory.

2.2 Methods: Representing Object Behavior

A method describes an action or computation associated with an object. A method may receive parameters, may return a value, or may simply perform an action.

```
class Student {
    String name;
    String studentId;
    double gpa;

    void displayInfo() {
        System.out.println(studentId + " - " + name + " - GPA: " + gpa);
    }

    boolean isPassed() {
        return gpa >= 5.0;
    }

    double calculateBonusGpa(double bonus) {
        return Math.min(10.0, gpa + bonus);
    }
}
```

Calling methods on an object:

```
Student s1 = new Student();
s1.name = "An";
s1.studentId = "23520001";
s1.gpa = 8.2;

s1.displayInfo();
System.out.println("Passed: " + s1.isPassed());
System.out.println("GPA after bonus: " + s1.calculateBonusGpa(0.3));
```

Method design: A method should represent a responsibility of the object. For example, calculating whether a student passes belongs naturally to Student because it uses the student's GPA.

2.3 Access Modifiers: Controlling Visibility

IT002 – Object-Oriented Programming

Access modifiers determine where a class member can be accessed. They are an important part of class design because an object should expose only what other code needs to use while keeping implementation details protected.

Java provides three explicit access modifiers - public, private, and protected - plus package-private access when no modifier is written.

Modifier	Same class	Same package	Subclass*	Other package
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	Limited through inheritance
package-private (no modifier)	Yes	Yes	No**	No
private	Yes	No	No	No

* protected is especially useful when inheritance is introduced. In this lab, students only need to recognize its purpose and basic visibility rules.

** A subclass in the same package can access package-private members because it is in the same package, not because it is a subclass.

Recommended beginner design: keep object data private and expose necessary operations through public constructors and public methods.

```
public class Student {  
    private String name;  
    private String studentId;  
    private double gpa;  
  
    public Student(String name, String studentId, double gpa) {  
        this.name = name;  
        this.studentId = studentId;  
        this.gpa = gpa;  
    }  
  
    public void displayInfo() {  
        System.out.println(studentId + " - " + name + " - GPA: " + gpa);  
    }  
  
    public boolean isPassed() {  
        return gpa >= 5.0;  
    }  
}
```

Because name, studentId, and gpa are private, code outside Student cannot directly execute statements such as s1.gpa = -100. The class controls how its own state is created and used. Getter/setter methods are a common way to provide controlled access, but full encapsulation is developed in a later lab.

3. Constructors: Initializing Objects Correctly

A constructor is a special member used to initialize an object when it is created. A constructor has the same name as the class, has no return type, and is called automatically by the new keyword.

Without a custom constructor, values may be assigned one by one after object creation. This works, but it is easy to forget a required field. A constructor allows an object to start in a meaningful state.

3.1 No-Argument Constructor

```
class Student {
    String name;
    String studentId;
    double gpa;

    Student() {
        name = "Unknown";
        studentId = "N/A";
        gpa = 0.0;
    }
}

Student s1 = new Student();
System.out.println(s1.name); // Unknown
```

3.2 Parameterized Constructor and the this Keyword

A parameterized constructor receives values when the object is created. When a parameter has the same name as an attribute, this.attribute refers to the attribute of the current object.

```
class Student {
    String name;
    String studentId;
    double gpa;

    Student(String name, String studentId, double gpa) {
        this.name = name;
        this.studentId = studentId;
        this.gpa = gpa;
    }
}

Student s1 = new Student("An", "23520001", 8.2);
Student s2 = new Student("Binh", "23520002", 6.7);
```

In this.name = name, the left side is the object attribute and the right side is the constructor parameter.

3.3 Simple Validation in a Constructor

A constructor can also protect the initial state of an object. For example, GPA should remain within a valid range.

```
Student(String name, String studentId, double gpa) {
    this.name = name;
    this.studentId = studentId;

    if (gpa >= 0.0 && gpa <= 10.0) {
        this.gpa = gpa;
    } else {
        this.gpa = 0.0;
    }
}
```

Scope note: This lab introduces access modifiers and the basic idea of protecting object state. Full encapsulation design, including systematic getter/setter use and stronger validation strategies, is developed later.

3.4 User-Defined Copy Constructor

Java does not provide a special language-level copy constructor in the same way as C++. However, a programmer can define a normal constructor that receives another object of the same class and copies its state. This pattern is commonly called a copy constructor.

```
public class Student {
    private String name;
    private String studentId;
    private double gpa;

    public Student(String name, String studentId, double gpa) {
        this.name = name;
        this.studentId = studentId;
        this.gpa = gpa;
    }

    // User-defined copy constructor
    public Student(Student other) {
        this.name = other.name;
        this.studentId = other.studentId;
        this.gpa = other.gpa;
    }
}
```

Creating an independent Student object using the copy constructor:

```
Student s1 = new Student("An", "23520001", 8.2);
Student s2 = new Student(s1);
```

Important distinction: `Student s2 = s1;` does not create a new Student object. It only copies the reference, so `s1` and `s2` refer to the same object. In contrast, `new Student(s1)` creates a new object and copies the selected field values.

IT002 – Object-Oriented Programming

```
Student s1 = new Student("An", "23520001", 8.2);
Student s2 = s1; // same object
Student s3 = new Student(s1); // new object with copied values
```

For fields such as int, double, boolean, and immutable String values, direct field copying is sufficient for this lab. When a class contains mutable object fields, lists, or arrays, students should later learn the difference between shallow copying and deep copying.

4. Complete Example: Class -> Object -> Method Calls

The following example combines the concepts learned so far.

```
class Student {
    private String name;
    private String studentId;
    private double gpa;

    public Student(String name, String studentId, double gpa) {
        this.name = name;
        this.studentId = studentId;
        this.gpa = (gpa >= 0 && gpa <= 10) ? gpa : 0;
    }

    public Student(Student other) {
        this.name = other.name;
        this.studentId = other.studentId;
        this.gpa = other.gpa;
    }

    public void displayInfo() {
        System.out.println(studentId + " - " + name + " - GPA: " + gpa);
    }

    public boolean isPassed() {
        return gpa >= 5.0;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("An", "23520001", 8.2);
        Student s2 = new Student("Binh", "23520002", 4.6);
        Student s3 = new Student(s1);

        s1.displayInfo();
        System.out.println("Passed: " + s1.isPassed());

        s2.displayInfo();
        System.out.println("Passed: " + s2.isPassed());

        s3.displayInfo();
    }
}
```

5. Overloading: One Name, Several Valid Forms

After students understand classes, methods, and constructors, overloading becomes easier to understand. Overloading means defining multiple methods or constructors with the same name but different parameter lists. Java selects the matching version at compile time.

Two overloaded methods must differ by at least one of the following: number of parameters, parameter data types, or order of parameter data types. Changing only the return type is not enough.

5.1 Method Overloading

Suppose a calculator needs to add two integers, two decimal numbers, or three integers. The operation is conceptually the same, so the same method name add can be reused.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator calc = new Calculator();
System.out.println(calc.add(2, 3));           // calls add(int, int)
System.out.println(calc.add(2.5, 3.1));       // calls add(double, double)
System.out.println(calc.add(1, 2, 3));        // calls add(int, int, int)
```

Invalid overloading example:

```
// ERROR: parameter list is identical
int calculate(int x) { return x * 2; }
double calculate(int x) { return x * 2.0; }
```

5.2 Constructor Overloading

Constructor overloading allows objects to be created with different amounts of initial information.

IT002 – Object-Oriented Programming

```
class Student {
    private String name;
    private String studentId;
    private double gpa;

    public Student() {
        this("Unknown", "N/A", 0.0);
    }

    public Student(String name, String studentId) {
        this(name, studentId, 0.0);
    }

    public Student(String name, String studentId, double gpa) {
        this.name = name;
        this.studentId = studentId;
        this.gpa = gpa;
    }
}

Student s1 = new Student();
Student s2 = new Student("An", "23520001");
Student s3 = new Student("Binh", "23520002", 7.5);
```

The call `this(...)` invokes another constructor in the same class. If used, it must be the first statement in the constructor.

6. A Practical OOP Workflow

For each exercise in this lab, follow the same sequence instead of starting from code immediately:

1. Read the problem and identify the main object(s).
2. List the attributes that describe each object.
3. List the methods that represent the object's responsibilities.
4. Decide which members should be public, private, protected, or package-private.
5. Decide how an object should be initialized and design constructor(s), including a copy constructor when object copying is required.
6. Implement the class.
7. In `main()`, create several objects with different values.
8. Call every required method and verify the output.
9. Use overloading only when the same conceptual operation needs different parameter lists.

7. EXERCISES

The original Lab 2 exercise set contained useful class-design problems, but some tasks introduced lists, relationships between classes, date processing, and sorting before those topics were formally developed. The revised set below keeps the core exercises aligned with this lab's learning outcomes and arranges them from basic to more challenging.

Exercise 1: Rectangle Class - Basic Class Design

Create a Rectangle class that represents a rectangle.

- Attributes: width and height (double). Declare object data with appropriate access modifiers (private is recommended).
- A no-argument constructor that creates a 1 x 1 rectangle.
- A parameterized constructor that receives width and height. Invalid non-positive values should be replaced by 1.
- Methods: area(), perimeter(), and displayInfo().
- In main(), create at least three Rectangle objects and call all methods.

Goal: practice identifying attributes, methods, constructors, and objects.

Exercise 2: Fraction Class - State Validation and Object Operations

Create a Fraction class.

- Attributes: numerator and denominator (int), preferably private. The denominator must never be 0.
- Provide a no-argument constructor representing 0/1, a parameterized constructor, and a copy constructor Fraction(Fraction other).
- Methods: simplify(), add(Fraction other), subtract(Fraction other), multiply(Fraction other), divide(Fraction other), and display().
- Arithmetic methods should return a new Fraction object instead of only printing the result.
- Test the class using at least two Fraction objects, then create one additional Fraction using the copy constructor and verify that it is a distinct object.

Goal: practice methods that receive and return objects, basic access control, and the user-defined copy-constructor pattern.

IT002 – Object-Oriented Programming

Exercise 3: Time Class - Maintaining a Valid Object State

Create a Time class that stores a time of day.

- Attributes: hour, minute, second.
- A no-argument constructor for 00:00:00 and a parameterized constructor. Validate hour in [0, 23] and minute/second in [0, 59].
- Methods: display(), addSeconds(int seconds), and subtractSeconds(int seconds).
- The resulting time must remain valid and wrap around a 24-hour day when necessary.
- Demonstrate cases such as 23:59:50 + 20 seconds and 00:00:10 - 20 seconds.

Goal: practice updating object state while preserving valid values. This improves the original exercise, which only added or subtracted hours.

Exercise 4: Circle Class - Geometry as an Object

Create a Circle class C(a, b, r) using three attributes: centerX, centerY, and radius.

- Constructors: a default unit circle centered at (0, 0), and a parameterized constructor.
- Methods: area(), perimeter(), and contains(double x, double y).
- contains(x, y) returns true when the point lies inside or on the circle.
- Create several circles and test several points in main().

Goal: practice translating a mathematical model into attributes and methods without introducing class composition too early.

Exercise 5: Dice Class - Object State and Repeated Method Calls

Create a Dice class that models a six-sided die.

- Attribute: value, storing the most recent roll (1-6).
- Constructor initializes value to 1.
- Method roll() randomly generates a new value from 1 to 6, updates the object state, and returns the value.
- In main(), roll one Dice object n times, count the frequency of each face, and print the experimental probability of each face.

Goal: practice repeated calls to an object method and observe how object state changes.

IT002 – Object-Oriented Programming

Exercise 6: Calculator - Method Overloading

Create a Calculator class to demonstrate method overloading.

- Implement add(int a, int b).
- Implement add(double a, double b).
- Implement add(int a, int b, int c).
- Implement max(int a, int b) and max(double a, double b).
- In main(), call every overloaded version and explain which method is selected for each call.

Goal: directly assess method overloading after students have already mastered ordinary methods.

Exercise 7: UITStudent - Constructor Overloading (Challenge)

Create a simplified UITStudent class. This challenge keeps the original student theme while removing date-list-sorting requirements that are outside the core scope of this lab.

- Attributes: studentId, fullName, major, and GPA. Use private for object data and public for operations intended to be called from Main.
- Provide three constructors: no arguments; studentId + fullName; and studentId + fullName + major + GPA.
- Use constructor chaining with this(...) where appropriate.
- Methods: displayInfo(), isPassed(), and academicClassification() using reasonable GPA thresholds specified in your report.
- Create at least five students using different constructors and display all results.

Goal: practice constructor overloading and object creation with different initial information.

Optional Extension: Author and Book

The original Author-and-Book problem is useful, but it introduces a relationship between two classes and List<Book>. It is better treated as an optional extension or moved to a later lab on object relationships and collections. If assigned here, students should already know ArrayList and how one object can contain references to other objects.

8. Submission Checklist

- For each exercise, clearly state the identified object/class, attributes, methods, constructor(s), and selected access modifiers before showing source code.
- Provide screenshots of source code and output.
- For each method, state its input, output, and purpose.
- Demonstrate all required constructors (including copy constructors where required) and overloaded methods with concrete test cases.
- Use meaningful class, variable, and method names and keep indentation consistent.